# Tres Bakery Chatbot

A03 — RAG + Tools Extension

Architecture Reference

This document provides a visual architecture diagram and detailed explanations of every key component in the A03 Tres Bakery chatbot system, covering the RAG retrieval pipeline, tool orchestration layer, and LLM inference engine.

| | |
|---|---|
| **Group members** | Irha · Areesha · Shaheera |
| **Assignment** | A03 — RAG + Tools Extension |
| **LLM model** | qwen3:1.7b via Ollama (local, CPU) |
| **Vector store** | ChromaDB · 60 docs · ~65 chunks |

## System Architecture Diagram

The diagram below shows the full request pipeline from the browser UI through the FastAPI server, Conversation Manager, RAG retrieval, tool orchestration, and LLM inference. Dashed lines indicate data flowing back into the LLM as enriched context after RAG retrieval and tool execution.

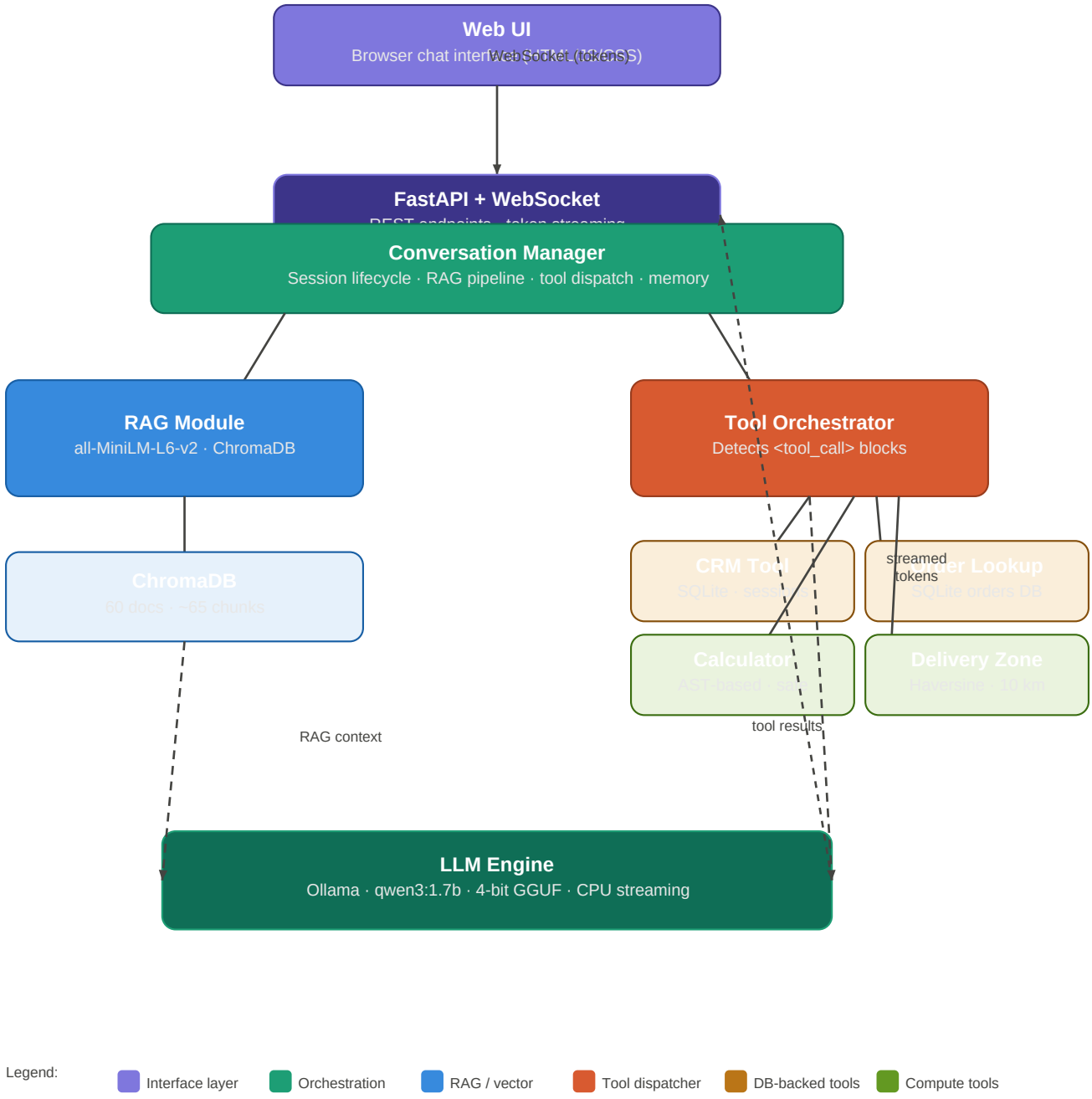


*Figure 1 — Tres Bakery Chatbot A03 full pipeline. Solid arrows = primary data flow. Dashed arrows = enriched context injection.*

# Key Component Explanations

INTERFACE LAYER

## Web UI

**Role:** The browser-based chat front-end that customers interact with.

- Single-page HTML/CSS/JavaScript application — no framework required.
- Connects to the backend over a persistent WebSocket and renders streamed tokens word-by-word for a responsive feel.
- Sends a JSON message containing an optional session_id and the user's text; receives incremental token events and a final done signal with latency metadata.

**Technologies:** Vanilla HTML · CSS · JavaScript

## FastAPI + WebSocket Server

**Role:** The network gateway — accepts connections, routes messages, and streams responses.

- Built on FastAPI (Python) with Uvicorn as the ASGI server.
- Exposes a WebSocket endpoint (WS /ws/chat) for real-time streaming and REST endpoints for session management, voice transcription, and synthesis.
- Passes each incoming message to the Conversation Manager and forwards streamed tokens back to the browser as they arrive.

**Technologies:** FastAPI · Uvicorn · Python 3.11+

ORCHESTRATION LAYER

## Conversation Manager

**Role:** The central coordinator — the 'brain' that sequences every step before a reply is generated.

- Maintains session state: conversation history, structured memory facts, and session metadata.
- On each turn: (1) extracts key facts from history via memory.py, (2) triggers RAG retrieval, (3) builds the enriched prompt, (4) streams the LLM response, and (5) post-processes any tool calls detected in the output.
- Implements a three-tier memory strategy — structured facts, last-6-turn verbatim history, and compressed older messages — to fit within the 4096-token context window.

**Technologies:** Python asyncio · conversation_manager.py · memory.py

RAG LAYER

## Retrieval Module (RAG)

**Role:** Grounds every answer in real bakery knowledge by fetching the most relevant document chunks before the LLM is called.

- Embeds the user query using all-MiniLM-L6-v2 (sentence-transformers, CPU, ~80 MB). An LRU cache of 256 entries drops repeated embedding time from ~80 ms to ~4 ms.
- Searches ChromaDB with cosine similarity and returns the top-5 chunks from ~65 chunks indexed from 60 bakery documents (menu, allergen policy, custom cake guides, delivery rules, catering, FAQs).
- Chunks are 512 tokens each with 64-token overlap (tiktoken cl100k_base). The retrieved context is injected into the system prompt before LLM inference.
- Total RAG overhead: ~110 ms cold, <10 ms cached — well within the 2-second target.

**Technologies:** rag.py · sentence-transformers · ChromaDB · tiktoken

TOOL LAYER

## Tool Orchestrator

**Role:** Enables the chatbot to take real actions by detecting tool invocations in the LLM output and dispatching them.

- Scans each streamed LLM output chunk for {...} JSON blocks.
- Parses the JSON, validates the tool name and arguments, and dispatches the correct async coroutine — each with a 5–8 s timeout via asyncio.wait_for.
- Re-prompts the LLM with the tool result appended as context so the model can incorporate the real data into its final reply.
- Handles malformed JSON gracefully — a known limitation of small quantised models.

**Technologies:** tool_orchestrator.py · Python asyncio

## CRM Tool

**Role:** Remembers returning customers across sessions using a local SQLite database.

- Supports three operations: get_user (retrieve profile), update_user (name, phone, email, preferences, allergy notes), and log_interaction (record a visit summary).
- Persists data to crm.db via aiosqlite so customer preferences survive server restarts.
- Invoked by the LLM when it detects a returning customer or needs to store a preference mentioned in conversation.

**Technologies:** crm_tool.py · SQLite · aiosqlite

## Calculator Tool

**Role:** Safely evaluates arithmetic expressions for order totals without using Python's eval().

- Parses expressions using Python's AST module — only whitelisted node types (numbers, operators, functions) are allowed; any unsafe construct raises an error.
- Handles expressions like '3 * 4.50 + 2 * 3.00' and returns the numeric result.
- Average latency under 5 ms — the fastest tool in the suite.

**Technologies:** calculator_tool.py · Python ast module

## Order Lookup Tool

**Role:** Lets customers check the status of their existing orders during a conversation.

- Queries a local SQLite orders database (orders.db) either by order ID or by customer name.
- Returns order details including items, status, and estimated delivery time.
- Average latency ~15 ms; data is simulated within conversation for demonstration purposes.

**Technologies:** order_lookup_tool.py · SQLite · aiosqlite

## Delivery Zone Tool

**Role:** Checks whether a given postcode falls within the bakery's delivery radius.

- Computes great-circle distance between the bakery's coordinates and the postcode's coordinates using the Haversine formula.
- Returns eligible or not eligible based on a 10 km radius threshold.
- Results are cached per postcode so repeat lookups cost nothing. Average latency under 5 ms.

**Technologies:** delivery_zone_tool.py · Haversine formula

INFERENCE LAYER

## LLM Engine

**Role:** Generates all natural-language responses using a locally hosted quantised language model.

- Runs qwen3:1.7b (4-bit GGUF quantisation) through Ollama — no cloud API required, all data stays local.
- Configured with temperature 0.4 (low variance), num_predict 600 (caps length), num_ctx 4096 (full context), and think=false (disables chain-of-thought for speed). Stop tokens prevent hallucinated turn continuations.
- Receives an enriched prompt: system instructions + structured memory + RAG chunks + tool results + recent conversation history.
- Streams tokens as soon as they are generated (~7 s time-to-first-token on CPU, 10–20 s total). Average latency is the bottleneck of the entire pipeline.

**Technologies:** llm.py · Ollama · qwen3:1.7b · 4-bit GGUF

## Performance Benchmarks

| Component / Step | Average latency | Target | Status |
|---|---|---|---|
| Query embedding (cold) | ~80 ms | < 200 ms | PASS |
| Query embedding (cached) | ~4 ms | < 200 ms | PASS |
| ChromaDB retrieval (top-5) | ~30 ms | < 200 ms | PASS |
| Total RAG overhead | ~110 ms | < 2000 ms | PASS |
| Calculator tool | < 5 ms | < 500 ms | PASS |
| Delivery zone tool | < 5 ms | < 500 ms | PASS |
| Order lookup tool | ~15 ms | < 500 ms | PASS |
| CRM tool | ~19 ms | < 500 ms | PASS |
| LLM inference (streaming) | 10 – 20 s | N/A (CPU) | — |
| End-to-end (typical query) | ~12 – 22 s | N/A (CPU) | — |